Université de Rouen Normandie

Faculté des Sciences et Techniques

Master 1 Génie Informatique et Logiciel

Projet de Théorie des Jeux

Implémentation du jeu Quoridor

Réalisé par :

HAMITOUCHE Lynda

OUACHOUR Amel

MHAREK Ayoub

Année Universitaire : 2025/2026

Table des matières

1 Introduction **3**

2 Objectifs du projet **3**

3 Environnement de développement **3**
3.1 Langage et bibliothèque . . . 3
3.2 Lancement du programme . . . 4
3.3 Arborescence du projet . . . 4

4 Manuel d'utilisation **4**
4.1 Écran d'accueil . . . 4
4.2 Modes de jeu disponibles . . . 6
4.3 Choix de l'algorithme . . . 6
4.4 Choix du niveau de difficulté . . . 7
4.5 Mode démonstration . . . 7
4.6 Déroulement d'une partie . . . 7
4.7 Déplacement du pion . . . 8
4.8 Placement d'un mur . . . 9
4.9 Fin de partie . . . 10
4.10 Règles du jeu . . . 11

5 Rappels sur les règles implémentées **12**
5.1 Position initiale . . . 12
5.2 Objectif . . . 12
5.3 Déplacements autorisés . . . 12
5.4 Règles de placement des murs . . . 13

6 Architecture générale du programme **13**
6.1 Fichier `main.py` . . . 13
6.2 Fichier `gui.py` . . . 13
6.3 Fichier `board.py` . . . 13
6.4 Fichier `ai_algorithms.py` . . . 14
6.5 Fichier `heuristics.py` . . . 14

7 Structures de données utilisées **14**
7.1 Représentation du plateau . . . 14
7.2 Hachage de Zobrist . . . 15
7.3 Table de transposition . . . 15

8 Description détaillée des fonctions principales **15**
8.1 Fonctions du modèle de jeu . . . 15
8.1.1 `is_blocked` . . . 15

8.1.2 `get_valid_pawn_moves` . . . 15
8.1.3 `is_valid_wall` . . . 15
8.1.4 `has_path` . . . 16
8.1.5 `make_move` et `undo_move` . . . 16
8.2 Fonctions heuristiques . . . 16
8.2.1 `shortest_path_distance` . . . 16
8.2.2 `evaluate_board` . . . 16

9 Fonctions d'évaluation choisies **16**
9.1 Distance au but . . . 16
9.2 Gestion des murs restants . . . 17
9.3 Progression et centralité . . . 17

10 Algorithmes de recherche utilisés **17**
10.1 MiniMax avec élagage Alpha-Beta . . . 17
10.2 SSS* . . . 17
10.3 Approfondissement itératif . . . 17
10.4 IA adaptative et hybridation d'algorithmes . . . 17

11 Étude de la stratégie de jeu **18**
11.1 Principes de la stratégie . . . 18
11.2 Implémentation dans l'IA . . . 18
11.3 Adaptation selon l'état du jeu . . . 19
11.4 Limites . . . 19

12 Conclusion **19**

1 Introduction

Le projet demandé consiste à implanter le jeu **Quoridor** avec une interface graphique agréable et pédagogique, en mobilisant plusieurs notions de théorie des jeux vues en cours : stratégie gagnante, élagage $\alpha\beta$, SSS*, heuristiques, tables de transposition et méthode de Zobrist. L'utilisateur doit pouvoir consulter les règles, observer une démonstration et jouer en choisissant un niveau de difficulté.

Quoridor se joue sur un plateau de **9 lignes et 9 colonnes**. Chaque joueur possède un pion et **10 barrières**. Le but est d'atteindre en premier le bord opposé du plateau. À chaque tour, un joueur choisit soit de déplacer son pion, soit de poser une barrière, sans jamais bloquer complètement tout chemin vers la ligne d'arrivée adverse. Les sauts au-dessus du pion adverse et les bifurcations latérales sont également autorisés.

Le programme étudié a été réalisé en **Python** avec **Pygame**. Il propose une interface graphique, plusieurs modes de jeu, plusieurs algorithmes d'intelligence artificielle, un système d'heuristique, une validation complète des murs et un hachage de Zobrist pour mémoriser efficacement les positions du jeu.

2 Objectifs du projet

Les objectifs couverts par l'application sont les suivants :

— implémenter les règles principales de Quoridor sur un plateau 9×9 ;

— proposer une interface graphique interactive et pédagogique ;

— permettre le jeu **joueur contre IA**, **joueur contre joueur** et proposer un mode de démonstration **IA contre IA** ; ;

— offrir plusieurs niveaux de difficulté ;

— intégrer plusieurs approches de recherche dans l'arbre de jeu ;

— utiliser une fonction d'évaluation heuristique pour guider l'IA ;

— exploiter des mécanismes de mémorisation des positions (table de transposition et hachage de Zobrist).

3 Environnement de développement

3.1 Langage et bibliothèque

Le projet est développé en **Python**. La dépendance explicitement visible dans le fichier `requirements.txt` est :

```
pygame-ce>=2.3.0
```

Le moteur graphique est donc **Pygame CE**, utilisé pour l'affichage, la gestion des événements, les sons et l'animation.

3.2 Lancement du programme

Le point d'entrée principal est le fichier `main.py`. Ce fichier initialise Pygame, lance une animation d'introduction, affiche le menu principal puis ouvre la fenêtre du jeu.

Pour exécuter le projet, on utilise les commandes suivantes :

```
pip install -r requirements.txt
python main.py
```

3.3 Arborescence du projet

L'organisation observée du projet est la suivante :

```
projet/
|-- docs/
| |-- rapport.pdf
|-- src/
| |-- ai_algorithms.py
| |-- board.py
| |-- gui.py
| `-- heuristics.py
|-- main.py
|-- requirements.txt
```

Le dossier `src` regroupe la logique principale du jeu, tandis que `main.py` joue le rôle de point d'entrée principal. Le dossier `docs` contient le rapport du projet.

4 Manuel d'utilisation

4.1 Écran d'accueil

Au lancement de l'application, une animation d'introduction s'affiche avant l'accès au menu principal. Cet écran présente le titre **QUORIDOR** ainsi que la mention du projet Master 1 GIL.

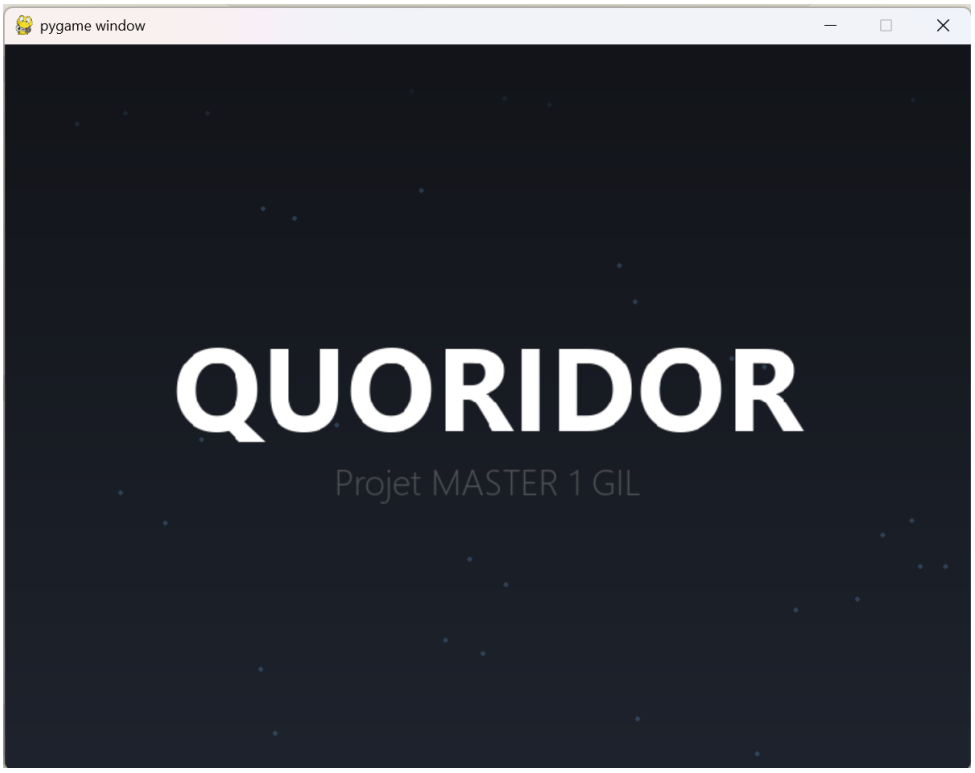


FIGURE 1 – Écran d'introduction au lancement du jeu

Après cette animation, l'utilisateur accède au menu principal comportant plusieurs boutons :

— **JOUER** : lance une partie ;

— **Mode** : permet d'alterner entre les modes disponibles ;

— **Algo** : permet de choisir l'algorithme de l'IA ;

— **Niveau IA** : permet de choisir le niveau ;

— **Voir une Démo** : lance un mode IA contre IA via une démonstration ;

— **Règles** : affiche un écran contenant les règles principales du jeu.

La figure suivante montre le menu principal de l'application.



FIGURE 2 – Menu principal du jeu

4.2 Modes de jeu disponibles

Le programme permet trois modes :

1. **PvP** : Deux joueurs humains s'affrontent sur le même écran ;
2. **PvAI** : joueur contre intelligence artificielle ;
3. **AIvAI** : démonstration automatique entre deux intelligences artificielles en cliquant sur le bouton "Voir une Démo".

Le mode **PvAI** est le mode par défaut dans le menu principal.

4.3 Choix de l'algorithme

Le bouton **Algo** du menu permet de faire défiler trois choix :

— **Alpha-Beta** ;

— **SSS*** ;

— **IA Adaptative**.

Plus précisément, l'IA adaptative utilise Alpha-Beta lorsque le nombre total de murs déjà posés est strictement inférieur à 5. À partir de 5 murs posés ou plus, elle bascule vers SSS*.

Ce choix s'explique par le fait qu'en début de partie, le plateau est encore ouvert avec un grand nombre de possibilités, ce qui rend l'algorithme Alpha-Beta plus efficace grâce à son

élagage. En revanche, lorsque plusieurs murs sont posés, le jeu devient plus contraint et les positions plus structurées, ce qui rend SSS* plus pertinente pour explorer les configurations les plus prometteuses.

4.4 Choix du niveau de difficulté

Le bouton **Niveau IA** fait varier la difficulté de **1 à 4**. Ce niveau est utilisé comme profondeur maximale de recherche transmise aux fonctions d'IA. Un niveau plus élevé implique donc une recherche potentiellement plus poussée et des temps de réflexion plus importants.

4.5 Mode démonstration

L'application permet également d'observer des exemples de parties grâce au bouton **Voir une Démo**. Ce mode lance une partie automatique entre deux intelligences artificielles (`AIvAI`).

Ces démonstrations sont générées dynamiquement. Leur déroulement dépend des paramètres choisis dans le menu, notamment l'algorithme sélectionné et le niveau de difficulté.

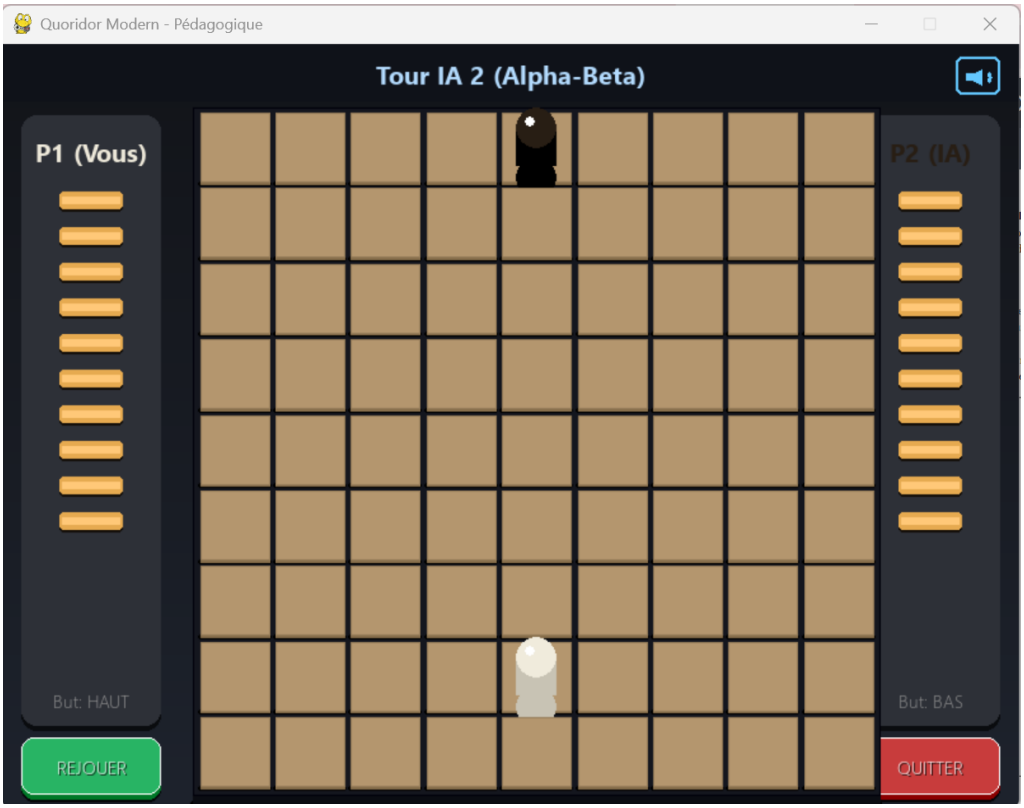


FIGURE 3 – Exemple de démonstration automatique AI contre AI

4.6 Déroulement d'une partie

Pendant la partie, la fenêtre affiche :

— le plateau de jeu en représentation de type 2.5D ;

— les deux pions ;

— les murs déjà posés ;

— le nombre de murs restants pour chaque joueur ;

— le statut du tour en cours ;

— une estimation heuristique de l'avantage courant ;

— des boutons de contrôle pour recommencer ou quitter.

— Une icône en haut à droite permet d'activer/désactiver la musique et les sons.

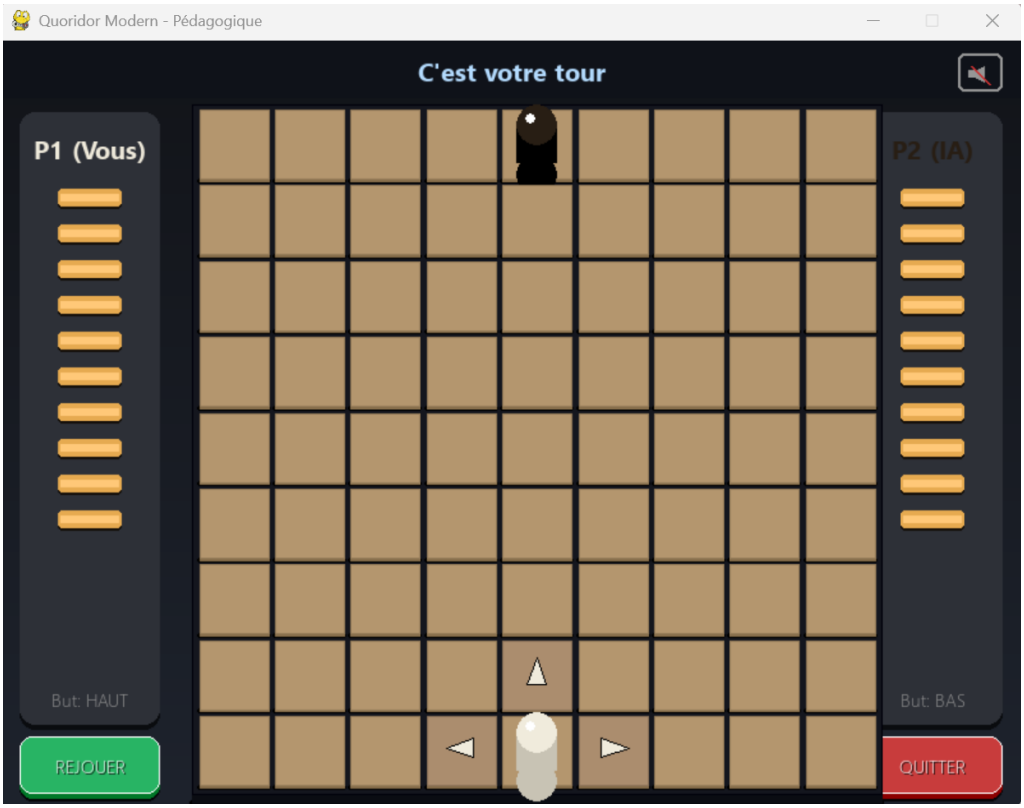


FIGURE 4 – Interface principale pendant une partie

4.7 Déplacement du pion

Le joueur humain peut déplacer son pion de deux manières :

1. **à la souris**, en cliquant sur une case valide ;
2. **au clavier**, avec les flèches directionnelles, si la case visée correspond à un mouvement autorisé.

Le programme met en évidence les déplacements possibles du pion et affiche des flèches directionnelles sur les cases autorisées.

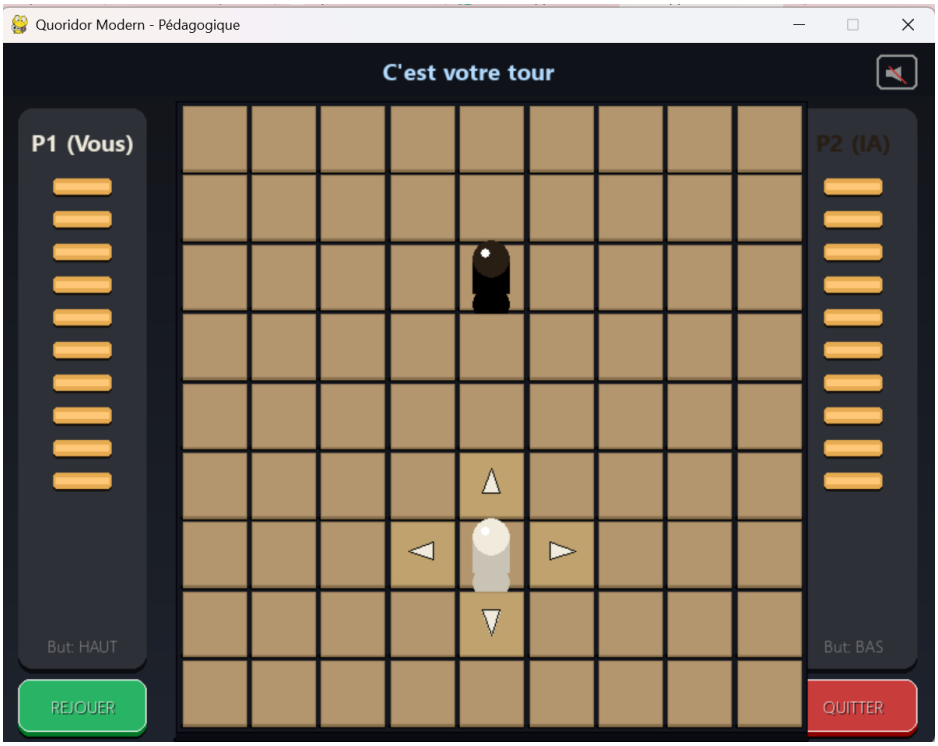


FIGURE 5 – Exemple de déplacement du pion avec mise en évidence des coups possibles

4.8 Placement d'un mur

Le joueur peut également placer un mur à la souris. Lors du survol du plateau, le programme détecte l'intersection la plus proche entre quatre cases et propose un mur **horizontal** ou **vertical** selon la position du curseur. Si le mur est légal, il est affiché en surbrillance avant validation par clic.

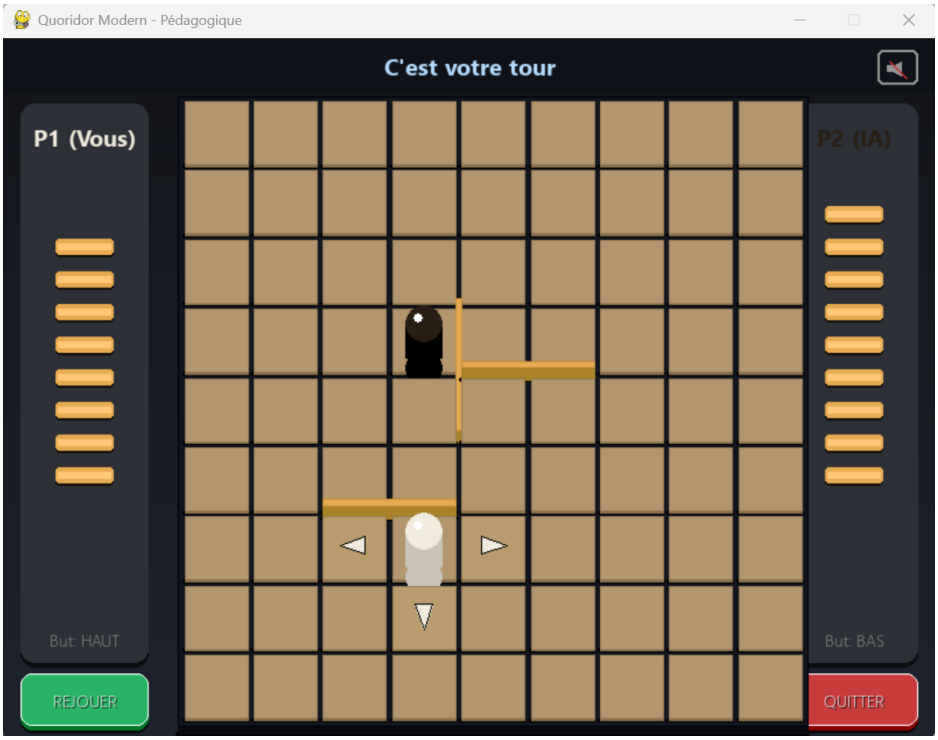


FIGURE 6 – Exemple de placement d'un mur

4.9 Fin de partie

La partie se termine dès qu'un joueur atteint sa ligne d'arrivée. Un écran de fin s'affiche alors avec :

— le résultat (victoire ou défaite) ;

— le vainqueur ;

— le nombre de coups joués ;

— la durée de la partie ;

— l'algorithme utilisé par l'IA si nécessaire.

Des effets sonores et visuels sont également présents lors de la victoire ou de la défaite.

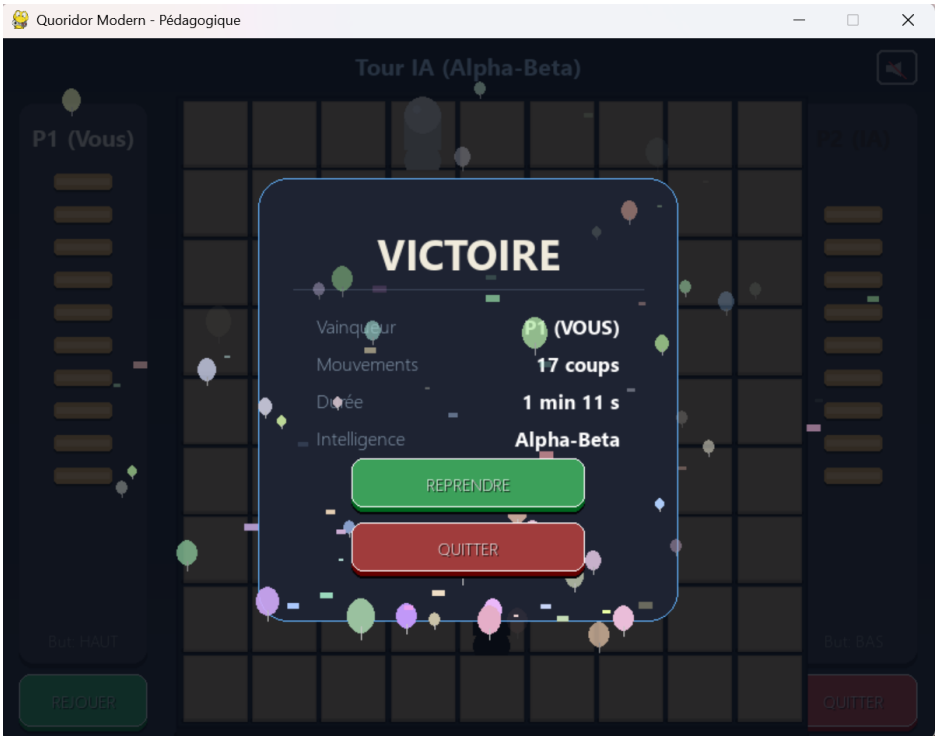


FIGURE 7 – Écran de fin de partie

4.10 Règles du jeu

L'application propose un écran dédié aux règles du jeu, accessible depuis le menu principal via le bouton **Règles**. Cet écran permet à l'utilisateur de consulter les principes de base du jeu Quoridor avant de commencer une partie.

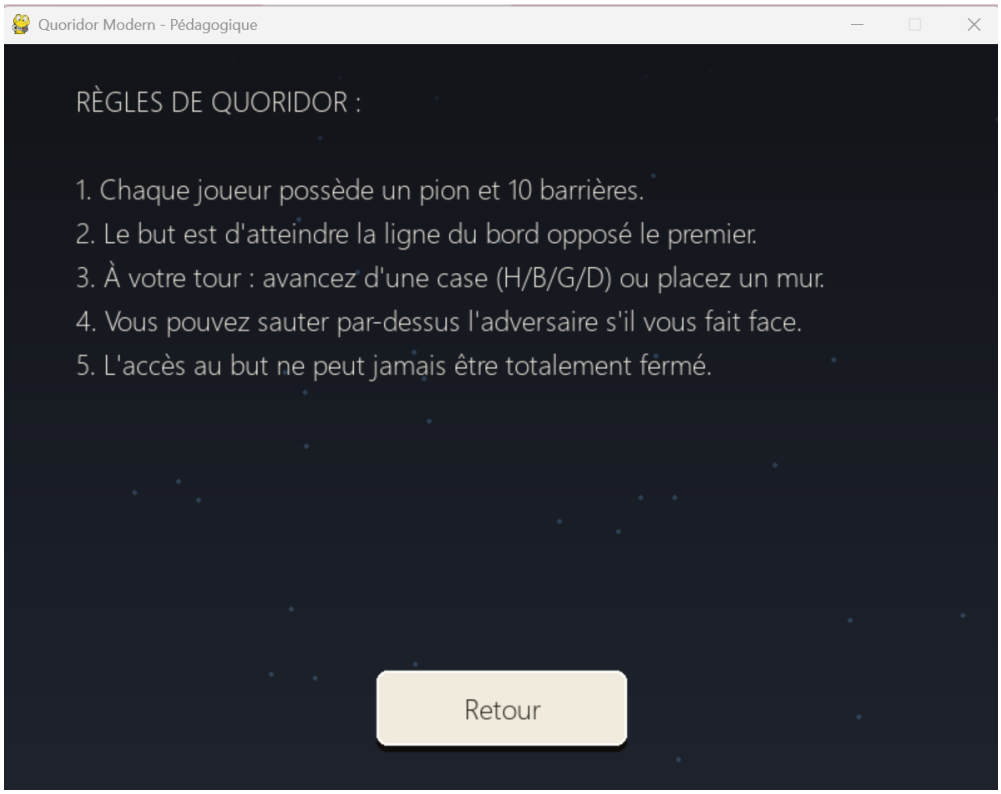


FIGURE 8 – Écran affichant les règles du jeu

5 Rappels sur les règles implémentées

Les règles observables dans le code sont conformes à l'énoncé fourni.

5.1 Position initiale

Le plateau est représenté par une grille $\mathbf{9} \times \mathbf{9}$. Le joueur 1 commence en bas au centre, soit en position `(8,4)`, et le joueur 2 commence en haut au centre, soit en position `(0,4)`.

5.2 Objectif

Le joueur 1 doit atteindre la ligne `0` et le joueur 2 doit atteindre la ligne `8`.

5.3 Déplacements autorisés

Le code autorise :

— les déplacements simples d'une case en haut, bas, gauche ou droite ;

— les sauts au-dessus du pion adverse lorsque les deux pions sont face à face sans mur entre eux ;

— les bifurcations latérales lorsque le saut direct est impossible à cause d'un mur ou du bord du plateau.

5.4 Règles de placement des murs

Le programme valide un mur selon plusieurs contraintes :

1. le joueur doit encore avoir des murs disponibles ;
2. le mur doit rester dans la zone autorisée ;
3. le mur ne doit ni chevaucher un mur existant, ni en croiser un autre ;
4. après placement, les deux joueurs doivent encore disposer d'au moins un chemin vers leur ligne d'arrivée.

6 Architecture générale du programme

Le projet est organisé en plusieurs fichiers.

6.1 Fichier `main.py`

`main.py` joue le rôle de point d'entrée principal. Il initialise les ressources, lance l'écran d'introduction, gère le menu principal, la sélection du mode de jeu, du niveau et de l'algorithme, puis instancie la classe `QuoridorGUI` pour démarrer la partie.

6.2 Fichier `gui.py`

`gui.py` contient la classe `QuoridorGUI`, qui gère :

— l'affichage du plateau et les objets graphiques ;

— la boucle de jeu ;

— les événements souris et clavier ;

— l'animation des pions ;

— le rendu de type 2.5D ;

— les effets sonores ;

— l'affichage des informations pédagogiques.

6.3 Fichier `board.py`

`board.py` contient la classe `QuoridorBoard`, qui constitue le modèle du jeu. Ce fichier gère :

— l'état complet du plateau ;

— la position des pions ;

— les ensembles de murs horizontaux et verticaux ;

— le nombre de murs restants ;

— le joueur courant ;

— la validation des déplacements ;
— la validation des murs ;
— la génération des coups légaux ;
— le hachage Zobrist de la position ;
— les opérations `make_move` et `undo_move` pour l'exploration de l'arbre.

6.4 Fichier `ai_algorithms.py`

Ce fichier regroupe les algorithmes de décision de l'IA. On y trouve :

— une table de transposition ;
— une fonction d'ordonnancement des coups ;
— MiniMax avec élagage Alpha-Beta ;
— Negamax ;
— SSS* ;
— une stratégie adaptative ;
— l'approfondissement itératif.

6.5 Fichier `heuristics.py`

Ce fichier contient les fonctions d'évaluation heuristique utilisées par l'IA pour attribuer un score à une position non terminale.

7 Structures de données utilisées

7.1 Représentation du plateau

La classe `QuoridorBoard` utilise les structures suivantes :

`Attribut`	Rôle
`pawns`	liste de deux couples `(ligne, colonne)` représentant la position des deux pions
`walls_h`	ensemble des murs horizontaux
`walls_v`	ensemble des murs verticaux
`walls_left`	liste de deux entiers représentant les murs restants
`turn`	entier valant 0 ou 1 selon le joueur courant
`hash`	valeur de hachage Zobrist de la position courante

Le choix des **ensembles** pour les murs permet une vérification rapide des collisions et des blocages.

7.2 Hachage de Zobrist

Le fichier `board.py` initialise une table de Zobrist contenant des nombres aléatoires 64 bits pour :

— chaque position possible de chaque pion ;

— chaque mur horizontal possible ;

— chaque mur vertical possible ;

— l'information de tour.

Le hachage de l'état est mis à jour de manière incrémentale dans `make_move`, ce qui évite de recalculer entièrement l'identifiant de la position à chaque coup.

7.3 Table de transposition

La table de transposition est implémentée par un dictionnaire Python. Chaque entrée mémorise :

— la profondeur ;

— le score ;

— un drapeau de type `EXACT`, `LOWERBOUND` ou `UPPERBOUND` ;

— le meilleur coup associé.

Cette structure permet d'éviter de recalculer certaines positions déjà rencontrées durant l'exploration de l'arbre de recherche.

8 Description détaillée des fonctions principales

8.1 Fonctions du modèle de jeu

8.1.1 `is_blocked`

Cette fonction détermine si un déplacement entre deux cases adjacentes est bloqué par un mur. Elle constitue le prédicat de base pour les règles de déplacement et les recherches de chemin.

8.1.2 `get_valid_pawn_moves`

Cette fonction génère tous les déplacements légaux du pion courant. Elle gère les mouvements simples, les sauts et les bifurcations latérales autour du pion adverse.

8.1.3 `is_valid_wall`

Cette fonction vérifie si un mur peut être posé légalement. Elle effectue à la fois les contrôles géométriques et la vérification de connectivité du plateau.

8.1.4 `has_path`

Cette fonction applique un parcours en largeur (BFS) pour savoir si un joueur donné peut encore atteindre sa ligne d'arrivée.

8.1.5 `make_move` et `undo_move`

Ces deux fonctions sont essentielles pour l'exploration de l'arbre de jeu. La première applique un coup et met à jour l'état du plateau ainsi que le hachage Zobrist ; la seconde annule ce coup à partir d'un état sauvegardé.

8.2 Fonctions heuristiques

8.2.1 `shortest_path_distance`

Cette fonction mesure la distance minimale entre un joueur et sa ligne de but grâce à un BFS. Elle sert de base à l'évaluation heuristique.

8.2.2 `evaluate_board`

La fonction d'évaluation combine plusieurs critères :

— la différence entre les distances minimales des deux joueurs vers leur but ;

— un bonus lié au nombre de murs restants ;

— un bonus de progression verticale ;

— un bonus de centralité pour éviter certaines oscillations sur les bords.

Le score retourné est positif lorsque la position est favorable au joueur maximisant passé en paramètre.

9 Fonctions d'évaluation choisies

La stratégie d'évaluation réellement observable dans le code repose sur une **heuristique combinée**. On peut l'exprimer ainsi :

$$\text{Score} = (\text{distance adverse} - \text{distance propre}) + \text{bonus murs} + \text{bonus progression/centralité}$$

9.1 Distance au but

Le premier critère mesure quel joueur est le plus proche de sa ligne d'arrivée. Ce critère est pertinent dans Quoridor car l'objectif final du jeu est strictement spatial : atteindre le bord opposé.

9.2 Gestion des murs restants

9.3 Progression et centralité

Le troisième critère favorise l'avancement réel du pion vers sa ligne d'arrivée et maintient une légère préférence pour les positions centrales.

10 Algorithmes de recherche utilisés

10.1 MiniMax avec élagage Alpha-Beta

L'algorithme principal de recherche est **MiniMax avec élagage Alpha-Beta**. Son fonctionnement général est le suivant :

— les positions terminales sont détectées via `check_winner` ;

— à profondeur nulle, le programme utilise `evaluate_board` ;

— les coups sont ordonnés avant exploration ;

— les bornes `alpha` et `beta` permettent de couper certaines branches inutiles ;

— la table de transposition peut fournir une valeur déjà mémorisée.

Le code prévoit également une pénalité de répétition basée sur un historique récent des positions, afin de décourager certaines boucles de jeu.

10.2 SSS*

Le projet contient également une fonction nommée `sss_star`, il s'agit d'une **implémentation** de SSS*. Concrètement, cette version évalue les coups immédiats et sélectionne le meilleur à l'aide d'une file de priorité.

10.3 Approfondissement itératif

La fonction `iterative_deepening` relance la recherche plusieurs fois avec des profondeurs croissantes. Le meilleur coup connu est mémorisé à chaque étape.

10.4 IA adaptative et hybridation d'algorithmes

La fonction `adaptive_search` sélectionne dynamiquement l'algorithme utilisé selon l'état du plateau. Cette approche hybride repose sur une analyse de la complexité structurelle du jeu :

— **Phase d'ouverture (0 à 4 murs)** : Le plateau est dit « ouvert ». Le facteur de branchement est maximal car les possibilités de placement de murs sont quasi-illimitées. Dans ce contexte, l'**élagage Alpha-Beta** associé à une recherche en profondeur (*Depth-First*) est optimal pour écarter rapidement les branches absurdes et privilégier une progression positionnelle fluide.

— **Phase labyrinthique (5 murs et plus)** : Dès que 5 barrières ou plus sont posées, la topologie du plateau se complexifie et des « goulots d'étranglement » apparaissent. Le jeu ne dépend plus d'une simple progression spatiale mais de la résolution de contraintes de cheminement. L'algorithme **SSS*** (recherche *Best-First*) devient alors plus pertinent : il utilise une file de priorité pour explorer en premier les branches les plus prometteuses. Il évite ainsi d'évaluer des séquences de coups qui, bien que profondes, sont rendues obsolètes par l'obstruction d'un passage critique.

L'utilisation de ce seuil empirique permet à l'IA d'être **tactique et rapide** en début de partie, puis **stratégique et précise** dès que le labyrinthe impose une analyse fine des trajectoires.

Le nom de l'algorithme actif est ensuite affiché dans l'interface.

11 Étude de la stratégie de jeu

Le jeu Quoridor présente une forte complexité combinatoire, rendant difficile l'élaboration d'une stratégie gagnante parfaite. Dans ce projet, nous avons adopté une approche basée sur une stratégie de jeu approchée, construite à partir d'heuristiques et d'algorithmes de recherche.

11.1 Principes de la stratégie

La stratégie mise en œuvre repose sur plusieurs principes fondamentaux :

— **Avancer efficacement vers l'objectif** : privilégier les déplacements réduisant la distance vers la ligne d'arrivée ;

— **Ralentir l'adversaire** : utiliser les murs pour allonger le chemin adverse ;

— **Maintenir des options de déplacement** : éviter de se bloquer soi-même en conservant plusieurs chemins possibles ;

— **Gérer les ressources** : conserver des murs pour les moments critiques de la partie.

11.2 Implémentation dans l'IA

Ces principes sont intégrés dans le programme à travers la fonction d'évaluation heuristique. Celle-ci prend en compte :

— la distance minimale au but (calculée par BFS) ;

— la différence de position entre les joueurs ;

— le nombre de murs restants ;

— la progression du pion vers la ligne d'arrivée.

L'utilisation de l'algorithme Alpha-Beta permet ensuite d'explorer les différentes possibilités et de choisir le coup le plus avantageux selon cette évaluation.

11.3 Adaptation selon l'état du jeu

Une stratégie adaptative a également été mise en place : en début de partie, lorsque le plateau est ouvert, l'algorithme Alpha-Beta est privilégié. Lorsque le nombre de murs augmente, une approche inspirée de SSS* est utilisée afin de mieux explorer les positions les plus prometteuses.

11.4 Limites

Cette stratégie ne garantit pas une victoire systématique, car elle repose sur une exploration limitée de l'arbre de jeu et sur une heuristique. Toutefois, elle permet à l'IA d'adopter un comportement cohérent et compétitif dans la majorité des situations.

12 Conclusion

Le programme étudié constitue une implémentation fonctionnelle et cohérente du jeu Quoridor en Python avec interface graphique. Il respecte les règles principales du jeu, propose plusieurs modes d'utilisation, intègre plusieurs approches d'intelligence artificielle et exploite des notions importantes de théorie des jeux et d'algorithmique : recherche arborescente, heuristiques, BFS, élagage Alpha-Beta, table de transposition et hachage de Zobrist.